

The Simplification Tax — End-to-End Notebook

Runs the entire pipeline (SFT → DPO/SimPO/PPO → eval → plots) on a single GPU. This notebook is self-contained — every cell can be skipped/re-run independently as long as the relevant checkpoint exists in `./results/`. Use it on **Colab Pro (A100)** or any A100 host.

Required env vars:

- `OPENAI_API_KEY` — for the GPT-4o pairwise judge.

0. Setup

```
In [1]: import subprocess, sys
def pip_install(pkg):
    subprocess.check_call([sys.executable, '-m', 'pip', 'install', '-q', pkg])
for pkg in ['transformers>=4.40', 'datasets', 'peft>=0.10', 'accelerate', 'c
    try:
        name = pkg.split('>=')[0].replace('-', '_')
        __import__(name)
    except Exception:
        pip_install(pkg)

import os, sys, json, math, time, copy, random
import numpy as np
import torch
sys.path.insert(0, os.getcwd())
device = 'cuda' if torch.cuda.is_available() else ('mps' if torch.backends.mps
print(f'device: {device}')
RESULTS = os.environ.get('RESULTS_DIR', './results')
os.makedirs(RESULTS, exist_ok=True)
TS = time.strftime('%Y%m%d_%H%M%S')
print(f'timestamp: {TS}')
```

```
/opt/homebrew/Caskroom/miniconda/base/envs/minitorch/lib/python3.13/site-pac
kages/tqdm/auto.py:21: TqdmWarning: IPProgress not found. Please update jupyt
er and ipywidgets. See https://ipywidgets.readthedocs.io/en/stable/user_inst
all.html
```

```
from .autonotebook import tqdm as notebook_tqdm
device: mps
timestamp: 20260512_222019
```

0.5. Local smoke test (CPU / MPS in <1 min)

The full pipeline (sections 1–7 below) is **Pythia-1B + LoRA on an A100**: the SFT pass alone is ~45 min and the rest needs a GPU and an OpenAI key. The published results in

results/ came from that pipeline.

This section is a runnable in-notebook demo that exercises **the same loss/data code paths** on commodity hardware (sshleifer/tiny-gpt2 , 4 examples, 4 training steps each) so you can confirm the project's code actually works without provisioning a GPU.

It runs:

- **SFT** → **DPO** → **SimPO** mini-training (a few gradient steps each), reusing
utils/data.py , utils/lora_utils.py , models/dpo.py ,
models/simpo.py
- A judge-free eval (diversity + length only) via eval_runner.evaluate_one

PPO is **not** in this smoke test (TRL ≥1.0 dropped PPOTrainer ; train_ppo.py targets the pinned trl>=0.7.10,<1.0 env documented in README.md).

```
In [2]: # --- 0.5(a) Smoke-test SFT on tiny-gpt2 -----
import os, sys, json, copy, time
import torch, torch.nn.functional as F, torch.optim as optim
from torch.utils.data import DataLoader
from transformers import AutoModelForCausalLM, AutoTokenizer

sys.path.insert(0, os.getcwd())
from utils.data import (load_tldr_sft, load_tldr_preferences,
                        SFTDataset, PreferenceDataset,
                        sft_collate, pref_collate, inject_label_noise,
                        _builtin_sft, _builtin_prefs)
from utils.lora_utils import add_lora, trainable_param_count
from models.dpo import get_logp_response, dpo_loss
from models.simpo import get_avg_logp_response, simpo_loss

SMOKE_MODEL = 'sshleifer/tiny-gpt2'
SMOKE_N = 4
SMOKE_STEPS = 4
SMOKE_MAXLEN = 384 # full TL;DR Reddit posts can be long; same as real tra
device = 'mps' if torch.backends.mps.is_available() else 'cpu'
print(f'[smoke] device={device} model={SMOKE_MODEL}')

tok = AutoTokenizer.from_pretrained(SMOKE_MODEL)
if tok.pad_token is None: tok.pad_token = tok.eos_token

# Use the short builtin TL;DR-style fixtures so the response fits within
# SMOKE_MAXLEN tokens after the prompt; real trl-lib/tldr posts can run >300
# tokens and would leave zero response mask under a tight max_len.
sft_examples = _builtin_sft(SMOKE_N)
pref_examples = _builtin_prefs(SMOKE_N)
print(f'[smoke] sft_examples={len(sft_examples)} pref_examples={len(pref_ex

# SFT
sft = AutoModelForCausalLM.from_pretrained(SMOKE_MODEL).to(device)
sft = add_lora(sft, r=4, alpha=8, dropout=0.0)
trainable, total = trainable_param_count(sft)
print(f'[smoke] SFT total={total/1e3:.1f}K trainable={trainable/1e3:.2f}K')
```

```

sft_ds = SFTDataset(sft_examples, tok, max_len=SMOKE_MAXLEN)
sft_loader = DataLoader(sft_ds, batch_size=2, shuffle=True,
                        collate_fn=lambda b: sft_collate(b, tok.pad_token_id))
opt = optim.AdamW([p for p in sft.parameters() if p.requires_grad], lr=1e-3)
sft_hist = []
sft.train()
for step, batch in enumerate(sft_loader):
    if step >= SMOKE_STEPS: break
    ids = batch['input_ids'].to(device); attn = batch['attention_mask'].to(device)
    out = sft(input_ids=ids, attention_mask=attn)
    sl = out.logits[:, :-1, :].contiguous().float()
    lab = ids[:, 1:].contiguous()
    sm = rm[:, 1:].contiguous().float()
    tok_loss = F.cross_entropy(sl.view(-1, sl.size(-1)), lab.view(-1), reduction='sum')
    loss = (tok_loss * sm).sum() / sm.sum().clamp(min=1.0)
    opt.zero_grad(); loss.backward(); opt.step()
    sft_hist.append({'step': step, 'loss': float(loss)})
    print(f'[smoke-sft] step={step} loss={loss.item():.4f} '
          f'(response tokens this batch: {int(sm.sum().item())})')

assert sft_hist[0]['loss'] > 0.5, "SFT loss too small - response mask probability too low"
print('[smoke] SFT mini-training OK ✓')

```

[smoke] device=mps model=sshleifer/tiny-gpt2

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.

[smoke] sft_examples=4 pref_examples=4

Loading weights: 0%| | 0/29 [00:00<?, ?it/s]

Loading weights: 100%|██████████| 29/29 [00:00<00:00, 19411.88it/s]

[transformers] GPT2LMHeadModel LOAD REPORT from: sshleifer/tiny-gpt2

Key	Status
transformer.h.{0, 1}.attn.masked_bias	UNEXPECTED

Notes:

- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.

/opt/homebrew/Caskroom/miniconda/base/envs/minitorch/lib/python3.13/site-packages/peft/tuners/lora/layer.py:2504: UserWarning: fan_in_fan_out is set to False but the target module is `Conv1D`. Setting fan_in_fan_out to True.

warnings.warn(

[smoke] SFT total=102.8K trainable=0.06K

[smoke-sft] step=0 loss=10.8234 (response tokens this batch: 27)

[smoke-sft] step=1 loss=10.8304 (response tokens this batch: 39)

[smoke] SFT mini-training OK ✓

/var/folders/m4/g0xp0q6n57b87xfl12tqwdhw0000gn/T/ipykernel_93822/1946135560.py:54: UserWarning: Converting a tensor with requires_grad=True to a scalar may lead to unexpected behavior. Consider using tensor.detach() first. (Triggered internally at /Users/runner/work/pytorch/pytorch/pytorch/torch/csrc/autograd/generated/python_variable_methods.cpp:837.)

sft_hist.append({'step': step, 'loss': float(loss)})

In [3]: # --- 0.5(b) Smoke-test DP0: SFT clone as both policy and ref -----
policy = AutoModelForCausalLM.from_pretrained(SMOKE_MODEL).to(device)

```

policy = add_lora(policy, r=4, alpha=8, dropout=0.0)
ref = AutoModelForCausalLM.from_pretrained(SMOKE_MODEL).to(device)
ref = add_lora(ref, r=4, alpha=8, dropout=0.0)
for p_ in ref.parameters(): p_.requires_grad_(False)
ref.eval()

pref_ds = PreferenceDataset(pref_examples, tok, max_len=SMOKE_MAXLEN)
pref_loader = DataLoader(pref_ds, batch_size=2, shuffle=True,
                        collate_fn=lambda b: pref_collate(b, tok.pad_token_
opt = optim.AdamW([p for p in policy.parameters() if p.requires_grad], lr=1e
dpo_hist = []
policy.train()
for step, batch in enumerate(pref_loader):
    if step >= SMOKE_STEPS: break
    ids_w = batch['ids_w'].to(device); attn_w = batch['attn_w'].to(device);
    ids_l = batch['ids_l'].to(device); attn_l = batch['attn_l'].to(device);
    logp_w = get_logp_response(policy, ids_w, attn_w, rm_w)
    logp_l = get_logp_response(policy, ids_l, attn_l, rm_l)
    with torch.no_grad():
        ref_w = get_logp_response(ref, ids_w, attn_w, rm_w)
        ref_l = get_logp_response(ref, ids_l, attn_l, rm_l)
    loss, ch_r, rj_r, acc = dpo_loss(logp_w, logp_l, ref_w, ref_l, beta=0.1)
    opt.zero_grad(); loss.backward(); opt.step()
    dpo_hist.append({'step': step, 'loss': float(loss), 'acc': float(acc),
                    'r_chosen': float(ch_r), 'r_rejected': float(rj_r)})
    print(f'[smoke-dpo] step={step} loss={loss.item():.4f} acc={acc.item():.
          f'r_w={ch_r.item():+.3f} r_l={rj_r.item():+.3f}')

# Sanity: step-0 DPO loss should be  $\sim \log 2 \approx 0.693$  (policy  $\approx$  ref initially)
assert abs(dpo_hist[0]['loss'] - 0.6931) < 0.05, f"unexpected step-0 DPO los
print('[smoke] DPO mini-training OK ✓ (step-0 loss matches log 2)')

# --- 0.5(c) Smoke-test SimPO -----
policy = AutoModelForCausalLM.from_pretrained(SMOKE_MODEL).to(device)
policy = add_lora(policy, r=4, alpha=8, dropout=0.0)
opt = optim.AdamW([p for p in policy.parameters() if p.requires_grad], lr=1e
simpo_hist = []
policy.train()
for step, batch in enumerate(pref_loader):
    if step >= SMOKE_STEPS: break
    ids_w = batch['ids_w'].to(device); attn_w = batch['attn_w'].to(device);
    ids_l = batch['ids_l'].to(device); attn_l = batch['attn_l'].to(device);
    _, _, avg_w = get_avg_logp_response(policy, ids_w, attn_w, rm_w)
    _, _, avg_l = get_avg_logp_response(policy, ids_l, attn_l, rm_l)
    loss, ch_r, rj_r, acc, margin = simpo_loss(avg_w, avg_l, beta=2.0, gamma
    opt.zero_grad(); loss.backward(); opt.step()
    simpo_hist.append({'step': step, 'loss': float(loss), 'acc': float(acc),
                      'r_chosen': float(ch_r), 'r_rejected': float(rj_r),
                      'margin': float(margin)})
    print(f'[smoke-simpo] step={step} loss={loss.item():.4f} acc={acc.item()
          f'margin={margin.item():+.3f}')

# Sanity: step-0 SimPO loss should be  $\sim -\log(\sigma(-1)) \approx 1.313$  since margin sta
import math
assert abs(simpo_hist[0]['loss'] - 1.3133) < 0.05, f"unexpected step-0 SimPO
print('[smoke] SimPO mini-training OK ✓ (step-0 loss matches  $-\log \sigma(-\gamma)$ )')

```

```
# Label noise path
noisy = inject_label_noise(pref_examples, flip_prob=0.5, seed=0)
print(f'[smoke] noise-injection OK ({len(noisy)} pairs)')
```

```
Loading weights: 0%|          | 0/29 [00:00<?, ?it/s]
Loading weights: 100%|████████| 29/29 [00:00<00:00, 64289.01it/s]
[transformers] GPT2LMHeadModel LOAD REPORT from: sshleifer/tiny-gpt2
Key                                     | Status      | |
-----+-----+-----+
transformer.h.{0, 1}.attn.masked_bias | UNEXPECTED | |
```

Notes:

- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.

```
Loading weights: 0%|          | 0/29 [00:00<?, ?it/s]
Loading weights: 100%|████████| 29/29 [00:00<00:00, 38310.18it/s]
[transformers] GPT2LMHeadModel LOAD REPORT from: sshleifer/tiny-gpt2
Key                                     | Status      | |
-----+-----+-----+
transformer.h.{0, 1}.attn.masked_bias | UNEXPECTED | |
```

Notes:

- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.

```
[smoke-dpo] step=0 loss=0.6933 acc=0.50 r_w=-0.000 r_l=-0.000
[smoke-dpo] step=1 loss=0.6921 acc=1.00 r_w=+0.000 r_l=-0.002
[smoke] DPO mini-training OK ✓ (step=0 loss matches log 2)
```

```
Loading weights: 0%|          | 0/29 [00:00<?, ?it/s]
Loading weights: 100%|████████| 29/29 [00:00<00:00, 54618.24it/s]
[transformers] GPT2LMHeadModel LOAD REPORT from: sshleifer/tiny-gpt2
Key                                     | Status      | |
-----+-----+-----+
transformer.h.{0, 1}.attn.masked_bias | UNEXPECTED | |
```

Notes:

- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.

```
[smoke-simpo] step=0 loss=1.3221 acc=0.00 margin=-0.012
[smoke-simpo] step=1 loss=1.3059 acc=1.00 margin=+0.010
[smoke] SimPO mini-training OK ✓ (step=0 loss matches -log σ(-γ))
[noise] flipped 2/4 labels (p=0.5)
[smoke] noise-injection OK (4 pairs)
```

```
In [4]: # --- 0.5(d) Smoke-test diversity / LC-win-rate metrics (no API calls) -----
        from models.eval_judge import (distinct_n, self_bleu,
                                         length_controlled_win_rate, likelihood_display)

        # Use the trained tiny SFT to generate a few completions on the same prompts
        # compare to a fresh (un-trained) tiny model as the "method".
        @torch.no_grad()
        def gen(model, prompts, n_new=24):
            out = []
            pad = tok.pad_token_id or tok.eos_token_id
```

```

    for p in prompts:
        enc = tok(p, return_tensors='pt', truncation=True, max_length=128).to(device)
        g = model.generate(**enc, max_new_tokens=n_new, do_sample=True,
                           temperature=0.7, top_p=0.95, pad_token_id=pad_token_id)
        txt = tok.decode(g[0][enc['input_ids'].shape[1]:], skip_special_tokens=True)
        out.append(txt)
    return out

prompts = [e['prompt'] for e in pref_examples]
method_gens = gen(sft, prompts)
baseline = AutoModelForCausalLM.from_pretrained(SMOKE_MODEL).to(device)
baseline = add_lora(baseline, r=4, alpha=8, dropout=0.0)
sft_gens = gen(baseline, prompts)
print('[smoke] generations done.')
print('  method[0]:', repr(method_gens[0][:80]))
print('  sft[0]    :', repr(sft_gens[0][:80]))

diversity = {
    'method_distinct_1': distinct_n(method_gens, 1),
    'method_distinct_2': distinct_n(method_gens, 2),
    'method_self_bleu_4': self_bleu(method_gens, 4, max_pairs=8),
    'sft_distinct_1': distinct_n(sft_gens, 1),
    'sft_distinct_2': distinct_n(sft_gens, 2),
    'sft_self_bleu_4': self_bleu(sft_gens, 4, max_pairs=8),
}
print('[smoke] diversity:', json.dumps(diversity, indent=2))

# Sample judgments to exercise the LC-win-rate code
sample_j = [{'winner': 'A'}, {'winner': 'B'}, {'winner': 'tie'}, {'winner': 'tie'}]
lens_a = [40, 30, 30, 60]; lens_b = [30, 30, 30, 30]
wr = length_controlled_win_rate(sample_j, lens_a, lens_b)
print('[smoke] LC win rate exerciser:', wr)

print('[smoke] displacement on DPO hist:', likelihood_displacement(dpo_hist))
print('[smoke] ALL CODE PATHS OK ✓ – full Pythia-1B pipeline lives in section 4.1')

```

```

Loading weights:  0%|          | 0/29 [00:00<?, ?it/s]
Loading weights: 100%|██████████| 29/29 [00:00<00:00, 39085.74it/s]
[transformers] GPT2LMHeadModel LOAD REPORT from: sshleifer/tiny-gpt2
Key                                     | Status      | |
-----+-----+-----+
transformer.h.{0, 1}.attn.masked_bias | UNEXPECTED | |

```

Notes:

- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.

```
[smoke] generations done.
  method[0]: 'HabitJD intermittentSherpressScene hauledreement ObservikenScene trilogy antibio'
  sft[0]    : 'hauledimura circumcisedmediately Observ004reement ONE pawn hauled autonomy Habit'
[smoke] diversity: {
  "method_distinct_1": 0.8135593220338984,
  "method_distinct_2": 1.0,
  "method_self_bleu_4": 0.06702758662741248,
  "sft_distinct_1": 0.9,
  "sft_distinct_2": 1.0,
  "sft_self_bleu_4": 0.06460607164391145
}
[smoke] LC win rate exerciser: {'raw_win_rate': 0.625, 'lc_win_rate': 0.25, 'alpha': 1.8865639866811423, 'mean_len_a': 40.0, 'mean_len_b': 30.0}
[smoke] displacement on DPO hist: {'final_r_chosen': 0.00015869137132540345, 'final_r_rejected': -0.0018775940407067537, 'mean_gap': 0.0008436202915618196, 'frac_positive_gap': 0.5, 'max_displacement': 0.0005264281935524195}
[smoke] ALL CODE PATHS OK ✓ – full Pythia-1B pipeline lives in sections 1–7 below.
```

1. Stage 1 — SFT [long-run]

Pythia-1B-deduped + LoRA on TL;DR. ~45 min on A100. Skip if you already have an SFT checkpoint in `results/sft/<ts>/model/`.

```
In [ ]: sft_out = f'{RESULTS}/sft/{TS}'
!python train_sft.py --output-dir $sft_out --bf16
SFT_CKPT = f'{sft_out}/model'
print(f'SFT checkpoint: {SFT_CKPT}')
```

2. Stage 2 — Train DPO [long-run]

```
In [ ]: dpo_out = f'{RESULTS}/dpo/default/{TS}'
!python train_dpo.py --output-dir $dpo_out --sft-checkpoint $SFT_CKPT --beta
DPO_CKPT = f'{dpo_out}/model'
```

3. Stage 3 — Train SimPO [long-run]

```
In [ ]: simpo_out = f'{RESULTS}/simpo/default/{TS}'
!python train_simpo.py --output-dir $simpo_out --sft-checkpoint $SFT_CKPT --
SIMPO_CKPT = f'{simpo_out}/model'
```

4. Stage 4 — Train PPO via TRL [long-run]

```
In [ ]: ppo_out = f'{RESULTS}/ppo/default/{TS}'
!python train_ppo.py --output-dir $ppo_out --sft-checkpoint $SFT_CKPT --kl-t
PPO_CKPT = f'{ppo_out}/model'
```

5. Evaluation harness [long-run]

Generates 500 completions per method, runs GPT-4o pairwise judge on 200 of them (both-orderings → LC win rate).

```
In [ ]: EVAL_OUT = f'{RESULTS}/eval'
os.makedirs(EVAL_OUT, exist_ok=True)
for name, ckpt in [('dpo_default', DPO_CKPT), ('simpo_default', SIMPO_CKPT),
                  !python eval_runner.py --checkpoint $ckpt --sft-checkpoint $SFT_CKPT \
                    --output-dir $EVAL_OUT --eval-name $name --n-eval 500 --n-judge 200
```

6. Plots and report tables

```
In [ ]: !python plotting.py --results-dir $RESULTS --out-dir $RESULTS/plots --eval-s
```

7. (Optional) β sweep and noise ablations

Each line below kicks one extra training run. Comment out to skip.

```
In [ ]: #  $\beta$  sweeps
for beta in [0.01, 0.05, 0.5, 1.0]:
    o = f'{RESULTS}/dpo/beta_{beta}/{TS}'
    !python train_dpo.py --output-dir $o --sft-checkpoint $SFT_CKPT --beta $
for beta in [0.5, 1.0, 5.0]:
    o = f'{RESULTS}/simpo/beta_{beta}/{TS}'
    !python train_simpo.py --output-dir $o --sft-checkpoint $SFT_CKPT --beta
for gamma in [0.5, 1.5, 2.0]:
    o = f'{RESULTS}/simpo/gamma_{gamma}/{TS}'
    !python train_simpo.py --output-dir $o --sft-checkpoint $SFT_CKPT --beta
# Label noise
for noise in [0.10, 0.25, 0.50]:
    o = f'{RESULTS}/dpo/noise_{noise}/{TS}'
    !python train_dpo.py --output-dir $o --sft-checkpoint $SFT_CKPT --beta 0
    o = f'{RESULTS}/simpo/noise_{noise}/{TS}'
    !python train_simpo.py --output-dir $o --sft-checkpoint $SFT_CKPT --beta
```